



Syllabus

FOUNDATION

Module 1 - Python Installation & Environment

1. Python introduction, features, use cases, versions
2. Python installation and environment setup
3. VS Code setup, Python extension, .py files
4. Python interpreter, running programs, pip
5. Virtual environment and first Python program

Module 2 - Variables & Data Types

1. Variables, naming rules, assignment, dynamic typing
2. Basic data types - int, float, str, bool, None
3. `type()` and basic type checking
4. Type conversion and type casting
5. Multiple variable assignment and basic memory concepts

Module 3 - Operators

1. Arithmetic and assignment operators
2. Comparison operators
3. Logical operators - and, or, not
4. Membership and identity operators - in, is
5. Operator precedence and expressions

Module 4 - Input & Output

1. `print()` and output formatting
2. `input()` and user input
3. Type conversion with user input
4. String formatting and f-strings
5. Escape characters and formatted output

PROGRAMMING LOGIC

Module 5 - Conditional Statements

1. `if`, `else`, and `elif`
2. Multiple conditions and nested conditions
3. Comparison and logical operators with conditions
4. Conditional expressions

Module 6 - Loops

1. `for` loop and `range()`
2. `while` loop and loop conditions
3. Nested loops
4. `break`, `continue`, and `pass`
5. Looping through strings, lists, and basic pattern programs

Module 7 - Strings

1. String creation, indexing, negative indexing, and slicing
2. String concatenation, repetition, and formatting
3. Important string methods - `upper()`, `lower()`, `strip()`, `replace()`
4. `split()`, `join()`, `find()`, `count()`, `startswith()`, `endswith()`
5. String-based problem solving - reverse, palindrome, character counting

DATA HANDLING

Module 8 - Lists

1. List creation, indexing, slicing, and updating
2. Adding/removing elements - `append()`, `insert()`, `remove()`, `pop()`
3. List methods - `sort()`, `reverse()`, `index()`, `count()`
4. Nested lists and iterating through lists
5. List comprehension and basic list problem solving

Module 9 - Tuple, Set & Dictionary

1. **Tuple**: creation, indexing, slicing, methods, tuple vs list
2. **Set**: creation, uniqueness, adding/removing, union/intersection/difference
3. **Dictionary**: key-value pairs, creation, access, update, delete
4. Dictionary methods - `keys()`, `values()`, `items()`, `get()`

REUSABLE PROGRAMMING

Module 10 - Functions

1. Creating and calling functions, parameters and arguments
2. Return values, multiple parameters, default and keyword arguments
3. Positional arguments, local/global variables
4. *args and **kwargs
5. Lambda functions and reusable function design

Module 11 - Exception Handling

1. Errors vs exceptions and common Python errors
2. `try` and `except`
3. Multiple exceptions, `else`, and `finally`
4. Common exceptions - `ValueError`, `TypeError`, `IndexError`, `KeyError`, `ZeroDivisionError`
5. `raise` and basic custom exception handling

Module 12 - File Handling

1. File creation, opening, closing, and file modes
2. Reading files - `read()`, `readline()`, `readlines()`
3. Writing/appending - `write()`, `writelines()`
4. `with open()` and file exception handling
5. Basic text and CSV file handling

Module 13 - Modules & Packages

1. Modules, `import`, and `from ... import`
2. Python built-in modules - `math`, `random`, `datetime`
3. Creating and using custom modules
4. Packages and basic project structure
5. `pip`, package installation, virtual environment, `requirements.txt`

OBJECT-ORIENTED PROGRAMMING

Module 14 - OOP Fundamentals

1. OOP concepts, classes and objects
2. Attributes, methods, and object creation

3. Constructor - `__init__()` and `self`
4. Instance variables and instance methods
5. Class variables and multiple objects

Module 15 - OOP Concepts

1. **Encapsulation** - public/private members and data protection
2. **Inheritance** - parent/child classes, reuse, `super()`
3. **Polymorphism** - method overriding and different behaviors
4. **Abstraction** - abstract classes and methods
5. Class methods, static methods, and basic OOP design

